

---

# Multi-Feature Riemannian Hypergraph for Online Test-Time Adaptation of Motor Imagery Brain-Computer Interface

---

Siqi Li<sup>1,2</sup>   Zhi Li<sup>3</sup>   Tong Liu<sup>3</sup>   Shuai Zhang<sup>3</sup>   Yanfei Jia<sup>4</sup>

Zhiqiang Yi<sup>4</sup>   Jue Xie<sup>3</sup>   Ni Ji<sup>5,2\*</sup>

<sup>1</sup>Peking University

<sup>2</sup>Chinese Institute for Brain Research, Beijing

<sup>3</sup>NeuCyber Neurotech

<sup>4</sup>Beijing Medical University

<sup>5</sup>Chinese Academy of Medical Sciences & Peking Union Medical College

## Abstract

In clinical motor imagery brain-computer interface (MI-BCI) decoding, cross-day transferability and online operation remain two critical challenges. Hypergraphs can improve transferability by capturing higher-order sample relationships, yet existing hypergraph-based methods for online emotion recognition neglect the cross-day benefits of Riemannian geometry widely adopted in EEG transfer learning. To bridge this gap, we propose the Multi-feature Riemannian Hypergraph (MRieHy), a framework tailored for online test-time adaptation in MI-BCI decoding that leverages Riemannian geometry to strengthen cross-day transferability. MRieHy first computes Riemannian means of covariance matrices from cross-day training data to align multi-day distributions. It then constructs a hypergraph over covariance matrices using Riemannian distance, complemented by a second hypergraph over deep features built with cosine similarity. The two hypergraphs are fused via adaptively learned combination weights, jointly optimized with the label projection matrices. During online testing, MRieHy maintains a first-in-first-out buffer of recent samples, performs Riemannian alignment on the buffered data, and decodes with the learned hypergraph. Extensive experiments on a private four-class ECoG dataset and two public four-class EEG datasets validate that MRieHy achieves notable performance gains over state-of-the-art baselines.

## 1 Introduction

Motor imagery brain-computer interfaces (MI-BCIs) represent a transformative neurotechnology that enables direct communication between the human brain and external devices through the decoding of imagined movement patterns without physical execution [17]. For individuals with spinal cord injuries, amyotrophic lateral sclerosis, or stroke-induced paralysis, MI-BCIs offer a vital channel for environmental interaction, communication, and neurorehabilitation [7, 3, 15].

Despite remarkable progress, the practical deployment of MI-BCIs faces a critical challenge: the inherent non-stationarity of neural signals across time. Brain signals fluctuate due to factors like varying attention and fatigue levels [23], typically necessitating a supervised calibration session

---

\*Corresponding author: [niji@cibr.ac.cn](mailto:niji@cibr.ac.cn)

before each use. This recurring requirement creates significant barriers for both patients with limited cognitive resources and the general public, who expect plug-and-play usability [34].

This challenge is compounded by real-time online operation, where signals stream continuously and demand immediate feedback. Prior work shows that even with calibration, many users still experience substantial performance degradation during online control, caused by the domain shift from offline calibration to real-time conditions [26]. Consequently, methods enabling effective domain adaptation from unlabeled streaming test-time data are essential for practical deployment.

Hypergraphs extend traditional graphs by modeling higher-order relationships among multiple entities simultaneously, going beyond simple pairwise connections [37]. By capturing such overall higher-order sample relationships, hypergraphs can improve generalization relative to simple feature extraction methods [33]. Recent studies demonstrate their effectiveness in online transfer learning for emotion recognition using multimodal physiological signals such as EEG [33, 18, 19].

However, these methods do not exploit the cross-day transfer benefits of Riemannian geometry, a common technique in EEG transfer learning. They also assume labeled data are available for online adaptation, which is rarely true in MI-BCI applications. Moreover, the drivers of cross-day variability may differ from those in emotion recognition, and MI-BCI systems demand responses on a much shorter timescale.

We propose a novel framework, Multi-feature Riemannian Hypergraph (MRieHy), for online test-time adaptation in MI-BCI decoding. MRieHy constructs hypergraphs from deep neural features and covariance matrices of EEG/ECoG signals, then fuses them for the final prediction. Inspired by Riemannian geometry for transfer learning, it adopts a Riemannian metric-based similarity for hypergraph construction and applies Riemannian alignment to align multi-day training data with streaming test data, directly mitigating cross-day distribution shifts. Our contributions include:

- We propose a Riemannian metric-based similarity for hypergraph construction, overcoming the limitations of conventional cosine-similarity-based hypergraph frameworks.
- MRieHy’s multi-feature architecture combines the complementary strengths of covariance representations, which excel with limited training data, and deep features, which become advantageous with larger datasets.
- To the best of our knowledge, this is the first work to validate online test-time adaptation on human MI-BCI ECoG data. We demonstrate that MRieHy outperforms state-of-the-art baselines on ECoG and EEG benchmarks, as confirmed by detailed ablation analyses.

## 2 Related Work

### 2.1 Riemannian Geometry for BCI

As the covariance matrices of spatiotemporal EEG signals are symmetric positive definite (SPD), they lie on the SPD manifold. On this manifold, Riemannian metrics appropriately define distances and mean functions, endowing BCIs with key advantages: equivalence between sensor and source spaces, robustness of the geometric mean to artifacts, and strong cross-day and cross-subject generalization capabilities [5].

Taking advantages of the Riemannian geometry, Barachant et al. [1] implement the minimum distance to mean (MDM) classification algorithm with Riemannian distance and Riemannian mean, as well as the linear discriminant analysis in Riemannian tangent space. Zanini et al. [34] improve the MDM algorithm by considering the dispersion of each class in addition to the mean of each class, and apply their algorithm to cross-day and cross-subject BCI classification. Riemannian geometry is further combined with graph to preserve the Riemannian locality of data structure for BCI transfer learning [32], and is joint with Long Short-Term Memory network for inter-day three-dimensional hand trajectories decoding from ECoG signals [8]. Relative to convolutional neural networks, Riemannian decoding methods perform comparably in within-day, cross-day, and cross-subject settings [6].

### 2.2 Online Test-Time Adaptation

Domain gaps frequently exist between training and testing data. At the same time, in real-world scenarios, obtaining calibration data from the same domain as the testing data is often difficult, and testing data typically arrives in an online stream. To overcome this challenge, online test-time adaptation methods have been proposed. These methods utilize only unlabeled testing data for adaptation during online inference [14]. This characteristic makes online test-time adaptation

particularly suitable for Brain-Computer Interfaces (BCIs), as a primary challenge in real-world BCI deployment is overcoming day-to-day instability without calibration [34].

Specifically, Riemannian alignment or Euclidean alignment of the covariance matrices of the EEG signals is used to increase the data similarity across days and subjects. In batch normalization in deep learning, the statistics are estimated and updated using the test-time data, instead of inheriting from training statistics directly [2, 30, 16, 29]. To update the model during testing, entropy minimization is used to increase the confidence of the model’s prediction. Meanwhile, marginal distribution regularization or source model guidance can be applied to prevent the model from assigning all test samples to a single class and from becoming overly confident in incorrect predictions, which could be the results of entropy minimization [30, 13, 20].

### 2.3 Hypergraph Learning

As a generalization of graph, hypergraph can represent complex relationship between objects other than pairwise relationship [37]. Excelling at multi-modal learning, hypergraph-based algorithms are applied to three-dimensional object classification with multiple view data [35], as well as emotional recognition with both personality and physiological signals [36].

Yalan et al. [33] adapt hypergraph-based algorithms for online cross-subject transfer learning in emotion recognition using electrocardiogram (ECG) signals by adding hyperedges that connect incoming samples from unknown subjects to a hypergraph constructed from data of known subjects. Furthermore, the multimodal learning capability of hypergraph-based methods has been exploited to fuse EEG, ECG, and galvanic skin response signals for online cross-subject emotion recognition [18, 19]. However, these methods require labeled data to update the hypergraph when performing online transfer learning for previously unseen subjects. The labeled test-time data may not be available in real-world BCI applications. Furthermore, their hypergraphs are constructed using k-nearest neighbors with cosine similarity, and thus do not leverage the advantages of Riemannian geometry in cross-day and cross-subject settings.

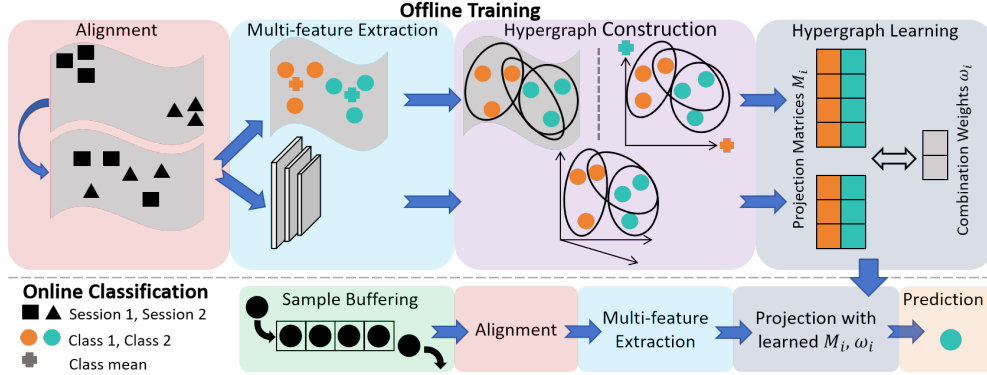


Figure 1: Schematic illustration of the Multi-feature Riemannian Hypergraph (MRieHy) framework.

## 3 Method

The Multi-feature Riemannian Hypergraph (MRieHy) framework, illustrated in Figure 1, enables robust online test-time adaptation for MI-BCI decoding. During offline training, it aligns multi-day data using Riemannian means and extracts both covariance matrices and deep features to construct two hypergraphs—utilizing Riemannian distance-based similarity for covariance matrices and cosine similarity for deep features. It then learns projection matrices and adaptive combination weights to fuse these graphs. For online classification, incoming unlabeled samples are buffered and aligned to the training distribution before features are extracted and projected using the pre-learned matrices and weights to predict labels.

### 3.1 Preliminary

In the  $N$ -class MI-BCI classification task, there is training data from  $K$  labeled source days  $\{D_S^k\}_{k=1}^K$  of a certain subject. In the  $k$ -th source day, the  $i$ -th sample  $\mathbf{X}_i^k \in \mathbb{R}^{C \times W}$  is obtained via channel-wise centering after sliding-window processing of a trial  $\mathbf{L}_j^k \in \mathbb{R}^{C \times T}$ , where  $C$  is the channel number,  $T$  is the time samples of the trial,  $W$  is window length, and  $W \leq T$ . It corresponds to

label  $y_i^k \in \{1, 2, \dots, N\}$ . Data from all source days is aligned and used to train hypergraph-based classifier  $f_S$ . When testing, a sequence of unlabeled samples  $\{\mathbf{X}_1^T, \mathbf{X}_2^T, \dots\}$  from target day  $D_T$  arrive in an online manner, and are dynamically stored in a first-in-first-out (FIFO) buffer with size  $b$ . Online test-time adaptation is then utilized to leverage the labeled knowledge implied in  $f_S$  to infer the labels of these samples in  $D_T$  under distribution shift [14].

### 3.2 Riemannian Alignment

Favored for its simplicity and effectiveness, covariance alignment functions as a standard transfer learning approach in BCI decoding [9, 34, 10]. The method assumes that differences between days can be eliminated by a whitening process with the mean of covariance matrices [25, 21, 34]. Specifically, the mean  $\bar{\mathbf{R}}^k$  of the covariance of the samples in day  $k$  in the sense of Euclidean (equation (1) left) or Riemannian geometry (equation (1) right) is first calculated:

$$\bar{\mathbf{R}}^k = \frac{1}{l} \frac{1}{W-1} \sum_{i=1}^l \mathbf{X}_i^k \cdot \mathbf{X}_i^{k\top}, \quad \bar{\mathbf{R}}^k = \arg \min_R \sum_{i=1}^l d_g^2(R, \frac{1}{W-1} \mathbf{X}_i^k \cdot \mathbf{X}_i^{k\top}) \quad (1)$$

where  $l$  is the sample number in day  $k$ , and  $d_g$  represents Riemannian distance. Then, each trial is aligned by:

$$\tilde{\mathbf{X}}_i^k = (\bar{\mathbf{R}}^k)^{-\frac{1}{2}} \cdot \mathbf{X}_i^k \quad (2)$$

The aligned sample  $\tilde{\mathbf{X}}_i^k \in \mathbb{R}^{C \times W}$  is used for the following decoding processes.

### 3.3 Riemannian Hypergraph Construction

For simplicity, the aligned sample  $\tilde{\mathbf{X}}_i^k$  is written as  $\mathbf{X}$  here. Using  $\mathbf{X}$ , multiple features are extracted for hypergraph learning. The covariance matrix  $\mathbf{F}_i^{co} \in \mathbb{R}^{C \times C}$  is computed as

$$\mathbf{F}_i^{co} = \frac{1}{W-1} \sum_{t=1}^W (\mathbf{X}_{:,t} - \bar{\mathbf{X}})(\mathbf{X}_{:,t} - \bar{\mathbf{X}})^\top \quad (3)$$

where  $\bar{\mathbf{X}} = \frac{1}{W} \sum_{t=1}^W \mathbf{X}_{:,t} \in \mathbb{R}^C$  denotes the temporal mean of the channel signals.  $\mathbf{F}_i^{co}$  can be seen as a point on the Riemannian manifold of symmetric positive definite (SPD) matrices [34]. We use  $\mathbf{f}_i^{co} \in \mathbb{R}^{C \cdot C}$  to represent the vectorized covariance matrix  $\mathbf{F}_i^{co}$ . Meanwhile, the aligned samples and the corresponding labels are used to train BaseNet, a lightweight yet powerful model for EEG decoding [30, 29]. The deep feature  $\mathbf{f}_i^{deep} \in \mathbb{R}^d$  before the final linear layer of BaseNet is extracted as another feature, where  $d$  represents the dimension of it.

Each of these two features serves as the basis for a hypergraph. To construct the hypergraph, we employ two distinct classes of methods to measure the similarity  $s(v_i, v_j)$  between hypergraph vertices  $v_i$  and  $v_j$  corresponding to samples  $\mathbf{X}_i$  and  $\mathbf{X}_j$ .

One class is based on the relationship between sample features directly. Belonging to this class is the cosine similarity (Cos), which can be applied to both covariance matrix  $\mathbf{f}_i^{co}$  and deep feature  $\mathbf{f}_i^{deep}$ :

$$s(v_i, v_j) = \frac{\langle \mathbf{f}_i, \mathbf{f}_j \rangle}{|\mathbf{f}_i| \cdot |\mathbf{f}_j|} \quad (4)$$

Two Riemannian versions to measure the relationship between sample features directly can be applied to the covariance matrix  $\mathbf{F}_i^{co}$ . When  $\mathbf{F}_i^{co}$  is projected to the tangent space at the Riemannian center of all covariance matrices, the tangent space cosine similarity (TanCos) is computed with

$$s(v_i, v_j) = \frac{\langle \tan(\mathbf{F}_i^{co}), \tan(\mathbf{F}_j^{co}) \rangle}{|\tan(\mathbf{F}_i^{co})| \cdot |\tan(\mathbf{F}_j^{co})|} \quad (5)$$

Another version is to use the Gaussian kernel similarity based on Riemannian distance  $d_g$  without projecting to tangent space (GauRie):

$$s(v_i, v_j) = \exp\left(-\frac{d_g^2(\mathbf{F}_i^{co}, \mathbf{F}_j^{co})}{2\sigma^2}\right), \quad \sigma = \frac{1}{M^2} \sum_{i=1}^M \sum_{j=1}^M d_g(\mathbf{F}_i^{co}, \mathbf{F}_j^{co}) \quad (6)$$

where  $M$  is the number of vertex pairs, and  $\sigma$  equals to the mean distance.

The second class is inspired from the MDM algorithm. We calculate the Euclidean means  $\bar{\mathbf{R}}_1^{Eu}, \dots, \bar{\mathbf{R}}_N^{Eu}$  or the Riemannian means  $\bar{\mathbf{R}}_1^{Rie}, \dots, \bar{\mathbf{R}}_N^{Rie}$  of the covariance matrices with labels  $1, \dots, N$ , using equations with identical form as (1). Then, for each covariance matrix  $\mathbf{F}_i^{co}$ , we compute the Euclidean distances to all Euclidean means (EuDM), the Riemannian distances to all Riemannian means (RieDM), or the Euclidean distances to all Riemannian means after projected to the tangent space at the mean of all covariance matrices (TanDM), forming  $N$ -dimension distance vector  $\mathbf{d}_i^{Eu} \in \mathbb{R}^N$ ,  $\mathbf{d}_i^{Rie} \in \mathbb{R}^N$ , or  $\mathbf{d}_i^{tan} \in \mathbb{R}^N$ . Finally, the similarity  $s(v_i, v_j)$  between vertices  $v_i$  and  $v_j$  is calculated using the cosine similarity of each type of the distance to means  $\mathbf{d}_i$  and  $\mathbf{d}_j$ .

Using the pairwise similarities from deep features and covariance matrix, we model the relationships among hypergraph vertices with two hypergraphs,  $\mathcal{G}_{deep} = (\mathcal{V}_{deep}, \mathcal{E}_{deep}, \mathbf{W}_{deep})$ , and  $\mathcal{G}_{co} = (\mathcal{V}_{co}, \mathcal{E}_{co}, \mathbf{W}_{co})$ , where  $\mathcal{V}$  represents the vertex set,  $\mathcal{E}$  represents the hyperedge set, and  $\mathbf{W}$  represents the diagonal matrix of hyperedge weight. We call  $\mathcal{G}_{co}$  *Riemannian hypergraph* when it is built with similarities calculated with equation (5) or equation (6), or calculated from  $\mathbf{d}_i^{Rie}$  or  $\mathbf{d}_i^{tan}$ , since they are Riemannian-based. In each hypergraph, we create a hyperedge for each vertex to connect it to its  $k$  nearest neighbors based on the similarities. This hyperedge captures the relationship among the samples it connects. Given the constructed hypergraph  $\mathcal{G}$ , the incidence matrix  $\mathbf{H}$  is obtained by computing each entry as:

$$\mathbf{H}(v, e) = \begin{cases} 1, & v \in e \\ 0, & v \notin e \end{cases} \quad (7)$$

In this framework, for any vertex  $v \in \mathcal{V}$  and hyperedge  $e \in \mathcal{E}$ , their degrees are defined respectively by

$$d(v) = \sum_{e \in \mathcal{E}} \mathbf{W}(e) \mathbf{H}(v, e), \quad \delta(e) = \sum_{v \in \mathcal{V}} \mathbf{H}(v, e) \quad (8)$$

Using these quantities, we construct two diagonal matrices,  $\mathbf{D}^v$  and  $\mathbf{D}^e$ , containing the vertex degrees and hyperedge degrees, respectively. The hyperedge weights are initialized uniformly as  $1/n_e$ , where  $n_e$  denotes the total number of hyperedges.

### 3.4 Multi-feature Hypergraph Learning

Let  $\mathbf{Z} = [\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_n]$  be the concatenation of all sample features, where  $n$  is the total sample number for training. Following [35], the goal of hypergraph learning is to learn a regularized matrix  $\mathbf{M}$  to project the feature matrix  $\mathbf{Z}$  to the label space to discriminate different categories.

To learn the projection matrix  $\mathbf{M}$  for a single hypergraph, a cost function  $\Psi$  is introduced, aggregating the hypergraph Laplacian regularizer  $\Omega(\mathbf{M})$ , the empirical loss  $\mathcal{R}_{emp}(M)$ , and a regularization term on  $\mathbf{M}$  given by  $\Phi(\mathbf{M})$ :

$$\Psi = \{\Omega(\mathbf{M}) + \lambda \mathcal{R}_{emp}(M) + \mu \Phi(\mathbf{M})\} \quad (9)$$

In the context of multi-feature hypergraphs, the cost function  $\bar{\Psi}$  for learning the projection matrices  $\mathbf{M}_h$  from all hypergraphs comprises two components: the aggregated learning costs across individual hypergraphs and a regularization term  $\Gamma$  applied to the combination weights  $\omega$ :

$$\bar{\Psi} = \sum_{h=1}^m \omega_h \{\Omega(\mathbf{M}_h) + \lambda \mathcal{R}_{emp}(\mathbf{M}_h) + \mu \Phi(\mathbf{M}_h)\} + \eta \Gamma(\omega) \quad (10)$$

where  $m$  denotes the number of hypergraphs (equaling 2 in this research), and  $h$  indexes the specific hypergraph. The projection matrices  $\mathbf{M}_h$  for all hypergraphs are learned by optimizing this cost function. The detailed process is described in the appendix.

### 3.5 Online Test-Time Adaptation

We deploy our multi-feature Riemannian hypergraph in an online setting where test samples arrive one by one and are stored in a FIFO buffer of size  $b$ . For each new sample, we align the buffer contents using Riemannian or Euclidean alignment (starting from the very first sample) for domain adaptation, and extract its covariance matrix  $\mathbf{f}_t^{co}$  and deep feature  $\mathbf{f}_t^{deep}$ . The final prediction is then obtained via

$$\hat{\mathbf{y}}_t = \omega_{deep} (\mathbf{f}_t^{deep})^\top \mathbf{M}_{deep} + \omega_{co} (\mathbf{f}_t^{co})^\top \mathbf{M}_{co} \in \mathbb{R}^N, \quad (11)$$

$$\hat{y}_t = \arg \max_c \hat{\mathbf{y}}_t(c) \quad (12)$$

$\mathbf{M}_1, \dots, \mathbf{M}_m$  in section 3.4 is specified as  $\mathbf{M}_{deep}$  and  $\mathbf{M}_{co}$  here, as well as  $\omega_1, \dots, \omega_m$ .

## 4 Experiments

### 4.1 Dataset

We apply our method to three MI-BCI datasets with two modalities: a private ECoG dataset ECoG128, and two public EEG datasets BCI Competition IV 2a [4] and Stieger2021 [24].

The ECoG128 dataset was acquired using a flexible electrode array of 128 channels, which was implanted epidurally over the primary motor and somatosensory cortices in the left hemisphere of a participant with tetraplegia due to spinal cord injury, covering an area of  $43 \times 43\text{mm}^2$ . The participant performed MI-BCI tasks with 4 classes. Data were collected across eight days. The number of trials per day ranged from 200 to 520, yielding a total of 2,840 trials. The neural signals (600 Hz sampling rate) are bandpass-filtered between 4 Hz and 100 Hz and subsequently resampled to 256 Hz, followed by a sliding window process with 1s window length and 1s stride. The samples after window sliding are then channel-wise centered, and are shuffled randomly.

BCI Competition IV 2a dataset [4] consists of EEG data recorded at 250 Hz using 22 electrodes from 9 subjects performing 4-class motor imagery task. It was recorded in two sessions on different days with 288 trials per session. The signals are low-pass filtered with a cutoff frequency of 40 Hz, and are channel-wise centered.

The Stieger2021 dataset [24] comprises 1000 Hz EEG recordings covering two- and four-class motor imagery tasks. We randomly selected 7 subjects from the dataset, each with 11 sessions recorded on separate days and 450 trials per session. Since the two-class conditions are contained within the four-class ones, all trials were framed as a four-class task. We selected signals from 24 channels centered over the motor cortex, low-pass filtered them at 200 Hz, resampled to 250 Hz, segmented them with a 1 s sliding window and 1 s stride, and finally applied channel-wise centering and random shuffling.

### 4.2 Experiment Setting

For the ECoG128 dataset, models were trained on Days 3–5 recordings with an 80/20 random train/validation split. Online evaluation with cumulative accuracy was conducted independently on held-out Days 6–8 data. To study the effect of training duration (Section 4.6), separate models were trained on subsets spanning 1 to 5 days: Day 5, Days 4–5, Days 3–5, Days 2–5, and Days 1–5.

For the BCI Competition IV 2a dataset, subject-specific models were trained on 80% of Day 1, validated on the remaining 20%, and tested online on all Day 2 recordings for cross-day generalization.

The Stieger2021 dataset followed the same pipeline as ECoG128: Days 1–8 were used for training (80/20 split), and Days 9–11 were held out for online evaluation with cumulative accuracy. Likewise, training length was varied from 1 day (Day 8) up to 8 days (Days 1–8).

We compare MRieHy with BaseNet [29, 30], Riemannian minimum distance to mean (RieMDM) [11], and an ensemble BaseNet+RieMDM, where RieMDM class probabilities are derived via softmax of the negative Riemannian distances from the test covariance to each class mean. Additionally, we evaluate against test-time adaptation methods including BN-adapt [22], Tent [27], PL [12], CoTTA [28], and T-TIME [13]. Euclidean cosine-similarity hypergraph with Euclidean alignment (EuHy)g and its multi-feature counterpart (MEuHy) are also included as hypergraph-based baselines.

### 4.3 Main Experimental Results

Table 1 compares MRieHy against 10 baselines for online test-time adaptation on the ECoG128, Stieger2021, and BCI Competition IV 2a datasets.

On the ECoG128 dataset, MRieHy achieves the highest average accuracy of  $64.1 \pm 0.9\%$  on test Days 6–8, surpassing the second-best method, BaseNet+RieMDM ( $62.7 \pm 0.8\%$ ), by 1.4% and outperforming other test-time adaptation methods by 1.9%–6.1%. EuHy ( $45.7 \pm 5.6\%$ ) and MEuHy ( $56.5 \pm 2.3\%$ ) perform drastically worse, highlighting the ineffectiveness of Euclidean cosine similarity and Euclidean alignment for SPD covariance matrices.

On the Stieger2021 dataset, MRieHy again achieves the highest average accuracy ( $54.1 \pm 0.4\%$ ), though it leads CoTTA ( $53.2 \pm 0.5\%$ ) by only 0.9%. Its advantages become clear upon closer inspection: it ranks first on four of the seven subjects—more than any other method—while remaining competitive on the rest. This consistent, subject-level robustness demonstrates MRieHy’s effectiveness as a test-time adaptation method.

| Method         | ECoG128         |                 |                 |                 | Stieger2021 (EEG) |                 |                 |                 |                 |                 |                 |                 |
|----------------|-----------------|-----------------|-----------------|-----------------|-------------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
|                | D6              | D7              | D8              | Avg             | Subj1             | Subj12          | Subj26          | Subj28          | Subj30          | Subj46          | Subj50          | Avg             |
| BaseNet        | 63.8±0.8        | 58.5±1.2        | 65.5±1.7        | 62.6±0.9        | 62.1±1.5          | 35.8±0.3        | 56.8±1.0        | 41.1±3.3        | <b>59.1±1.7</b> | 63.4±1.8        | 49.1±0.9        | 52.5±0.1        |
| RieMDM         | 62.0±0.4        | 56.4±1.1        | 64.4±0.4        | 60.9±0.6        | 48.4±0.3          | 36.8±0.5        | 52.1±0.5        | 37.8±0.1        | 42.0±0.3        | 50.4±0.3        | 39.1±0.1        | 43.8±0.2        |
| BaseNet+RieMDM | 63.8±0.9        | 58.7±1.1        | 65.7±1.5        | 62.7±0.8        | <b>62.2±1.4</b>   | 36.1±0.2        | 57.0±1.0        | 41.4±3.4        | 58.9±1.7        | 63.5±1.8        | 49.2±0.8        | 52.6±0.1        |
| BN-adapt       | 62.5±0.1        | 56.9±1.2        | 64.3±1.1        | 61.2±0.3        | 54.3±0.3          | 34.9±1.1        | 55.9±0.4        | 42.1±1.4        | 52.8±2.1        | 56.6±1.0        | 45.5±1.2        | 48.9±0.7        |
| Tent           | 59.0±0.3        | 54.0±1.1        | 61.0±1.4        | 58.0±0.8        | 49.4±0.8          | 32.1±0.9        | 50.6±0.8        | 39.5±1.6        | 48.7±0.8        | 52.6±0.7        | 41.1±1.5        | 44.9±0.6        |
| PL             | 63.0±0.7        | 57.9±1.0        | 64.0±1.4        | 61.6±0.5        | 55.2±0.3          | 35.5±1.2        | 56.8±0.4        | 42.7±1.4        | 53.6±1.8        | 57.1±0.9        | 45.8±1.1        | 49.5±0.7        |
| CoTTA          | 63.0±1.1        | 58.1±1.5        | 65.2±1.0        | 62.1±0.5        | 61.9±1.1          | 35.6±1.0        | 57.8±3.7        | 43.3±3.8        | 58.5±2.5        | 64.5±1.6        | <b>50.6±1.0</b> | 53.2±0.5        |
| T-TIME         | 63.8±0.7        | 58.3±0.5        | 64.5±1.1        | 62.2±0.3        | 55.6±0.4          | 35.7±1.1        | 57.0±0.2        | 43.9±1.6        | 53.6±2.0        | 57.5±0.9        | 46.3±1.3        | 49.9±0.7        |
| EuHy           | 49.2±5.6        | 44.6±5.7        | 43.4±5.6        | 45.7±5.6        | 26.6±1.1          | 23.7±0.4        | 36.7±1.2        | 16.9±0.0        | 25.7±1.9        | 27.8±1.7        | 29.9±0.4        | 26.8±0.7        |
| MEuHy          | 60.6±2.3        | 54.1±1.7        | 54.9±3.2        | 56.5±2.3        | 59.9±1.5          | 34.4±0.5        | 58.9±1.2        | 40.4±1.4        | 54.4±2.1        | 60.5±2.0        | 48.4±1.1        | 51.0±0.1        |
| MRieHy         | <b>65.7±0.5</b> | <b>59.5±0.9</b> | <b>67.2±1.8</b> | <b>64.1±0.9</b> | 61.9±0.3          | <b>37.5±0.7</b> | <b>60.0±0.8</b> | <b>45.9±0.8</b> | 58.2±1.3        | <b>65.3±1.7</b> | 49.6±0.3        | <b>54.1±0.4</b> |

| Method         | BCI Competition IV 2a (EEG) |                 |                 |                 |                 |                 |                 |                 |                 |                 |
|----------------|-----------------------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
|                | Subj1                       | Subj2           | Subj3           | Subj4           | Subj5           | Subj6           | Subj7           | Subj8           | Subj9           | Avg             |
| BaseNet        | 74.0±4.9                    | 58.0±2.7        | 82.9±3.3        | 52.3±9.3        | 71.4±1.7        | 50.7±2.5        | 84.4±3.7        | 70.8±1.8        | 69.1±4.8        | 68.2±1.4        |
| RieMDM         | 63.4±1.3                    | 48.4±1.1        | 68.6±1.6        | 54.9±0.9        | 40.7±0.7        | 43.6±1.2        | 62.3±1.1        | 75.3±0.9        | 70.4±3.2        | 58.6±0.2        |
| BaseNet+RieMDM | 74.4±4.1                    | 58.3±3.0        | <b>83.1±3.7</b> | 52.3±9.1        | <b>71.6±1.6</b> | 51.6±2.5        | 84.8±3.5        | 70.7±1.1        | 70.8±4.2        | 68.6±1.3        |
| BN-adapt       | 69.7±2.8                    | 57.3±0.9        | 76.9±7.9        | 46.5±8.1        | 70.3±3.6        | 51.7±3.3        | 79.7±11.4       | 64.8±3.0        | 64.8±1.6        | 64.6±2.8        |
| Tent           | 63.2±1.9                    | 51.7±2.4        | 71.6±5.1        | 45.1±6.9        | 65.3±1.6        | 50.0±0.3        | 74.9±8.8        | 59.5±0.7        | 60.0±2.1        | 60.1±0.9        |
| PL             | 69.1±3.3                    | 57.8±1.4        | 76.9±6.7        | 46.5±10.3       | 70.4±3.5        | 51.0±3.3        | 79.7±10.6       | 64.8±2.5        | 64.4±3.1        | 64.5±2.4        |
| CoTTA          | 70.4±3.7                    | 58.0±0.6        | 76.5±7.3        | 45.8±9.0        | 69.9±4.2        | 52.2±3.0        | 79.6±10.4       | 64.0±3.2        | 65.3±2.6        | 64.6±3.1        |
| T-TIME         | 69.3±3.7                    | 57.8±1.8        | 76.7±6.9        | 45.4±8.6        | 70.8±3.3        | 51.3±3.0        | 79.4±10.8       | 64.9±1.4        | 65.3±1.8        | 64.5±2.3        |
| EuHy           | 25.5±0.2                    | 25.0±0.0        | 28.5±5.4        | 25.0±0.3        | 24.7±0.6        | 25.5±0.8        | 25.0±0.0        | 25.0±0.0        | 29.3±7.4        | 25.9±0.7        |
| MEuHy          | 35.5±3.3                    | 29.9±3.1        | 45.8±7.6        | 47.6±5.7        | 70.1±3.1        | 39.6±1.0        | 35.8±0.3        | 34.8±7.3        | 52.5±7.7        | 43.5±1.6        |
| MRieHy         | <b>74.4±2.6</b>             | <b>60.3±2.9</b> | <b>83.0±2.8</b> | <b>57.6±3.5</b> | 71.4±2.0        | <b>54.6±4.5</b> | <b>85.8±2.7</b> | <b>77.3±2.4</b> | <b>76.6±3.8</b> | <b>71.2±0.9</b> |

Table 1: Comparison between MRieHy and the baseline methods on three datasets

On the BCI Competition IV 2a dataset, MRieHy continues to lead with an average accuracy of 71.2±0.9% over 9 subjects, surpassing BaseNet (68.2±1.4%), BaseNet+RieMDM (68.6±1.3%), and RieMDM (58.6±0.2%) by 2.6%–12.6%, and outperforming the other test-time adaptation methods by 6.6%–11.1%. EuHy (25.9±0.7%) and MEuHy (43.5±1.6%) again severely underperform.

MRieHy’s consistent outperformance confirms its core advantages: Riemannian metric-based similarity and alignment better capture SPD manifold properties, and multi-feature fusion (deep + covariance) via hypergraphs leverages complementary information more effectively than the simple ensemble of BaseNet and RieMDM.

#### 4.4 Ablation Study

We conducted an ablation study on all datasets to analyze the individual components of the MRieHy framework. Table 2 details the results for different alignment methods, feature combinations, and similarity strategies.

**Riemannian Similarity Outperforms Euclidean Methods.** Our results demonstrate a clear advantage of Riemannian geometry-based similarity measures over Euclidean methods. On all three datasets, Riemannian methods (TanCos, GauRie, RieDM, TanDM) consistently outperform Euclidean approaches (Cos, EuDM). While performance differences among Riemannian methods remain relatively small (within 3%), suggesting that properly leveraging the SPD manifold geometry matters more than the specific Riemannian similarity implementation.

**Riemannian Alignment Better Preserves Covariance Geometry.** Comparing the 6th and 7th rows of Table 2 reveals the importance of Riemannian alignment. When replacing Riemannian alignment with Euclidean alignment, performance drops by 1.1%–9.4% on three datasets. This decrease confirms that Riemannian alignment better preserves the geometric structure of covariance matrices across different days, effectively mitigating distribution shifts in online test-time adaptation.

**Multi-feature Fusion Delivers Critical Gains.** Across all three datasets, the full MRieHy model consistently outperforms versions that rely on only one feature type. Replacing multi-feature fusion with covariance features alone or deep features alone causes accuracy drops ranging from 4.2% to 15.8%. This consistent finding demonstrates that both feature types contribute complementary

| Align | Co | Sim    | Deep | ECoG128         | BCI Comp        | Stieger2021     |
|-------|----|--------|------|-----------------|-----------------|-----------------|
| Rie   | ✓  | Cos    | ✓    | 59.9±2.2        | 45.1±1.2        | 52.0±0.3        |
| Rie   | ✓  | TanCos | ✓    | 64.1±1.0        | <b>71.2±0.9</b> | <b>54.1±0.3</b> |
| Rie   | ✓  | GauRie | ✓    | 63.3±0.9        | 68.8±1.5        | 52.5±0.4        |
| Rie   | ✓  | EuDM   | ✓    | 59.6±2.1        | 44.8±1.1        | 52.0±0.2        |
| Rie   | ✓  | RieDM  | ✓    | 63.3±0.9        | 68.8±1.5        | 52.5±0.3        |
| Rie   | ✓  | TanDM  | ✓    | <b>64.1±0.9</b> | <b>71.2±0.9</b> | 54.1±0.4        |
| Eu    | ✓  | TanDM  | ✓    | 63.0±0.4        | 70.1±1.2        | 44.7±0.1        |
| Rie   | ✓  | TanDM  | ×    | 59.9±0.1        | 55.8±1.0        | 46.6±0.1        |
| Rie   | ×  | -      | ✓    | 59.8±0.9        | 66.3±1.0        | 51.1±0.1        |

Table 2: Ablation study of different alignment methods, features (Co&Deep), and similarity computation (Sim) for covariance matrix

information essential for robust decoding, and their fusion yields substantially better cross-day generalization than either feature alone.

#### 4.5 Number of Days Included in a Hyperedge

We count the distinct days per hyperedge in the covariance-based hypergraph component. On both ECoG128 and Stieger2021, hypergraphs built with TanDM contain more distinct days per hyperedge than those using cosine similarity, while the number of samples per hyperedge stays constant (Figure 2, Figure D.1). This suggests that Riemannian distance-based similarity better aligns samples across days, likely explaining MRieHy’s superior performance.

Figure 3 shows the neighbor vertices and incident hyperedges for the same sample, taken from Subject 1 of the Stieger2021 dataset, in the covariance-based hypergraph component built with cosine similarity and with TanDM. In the cosine-similarity case 19 of 31 neighbor vertices belong to Day 4—the same day as the target vertex. The TanDM case contains only 6 of 17 neighbor vertices from Day 4; the remaining vertices are same-class (Right Hand) samples from various days. This intuitively supports that Riemannian distance-based similarity helps hyperedges aggregate same-class vertices across different days.

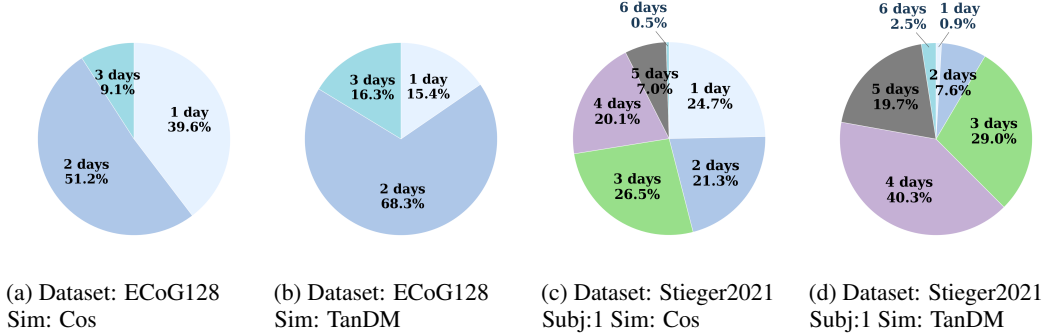


Figure 2: Percentage of distinct days per hyperedge in the covariance-based hypergraph component

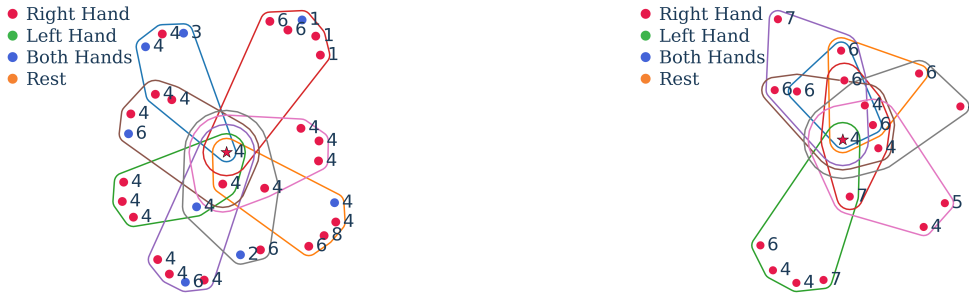


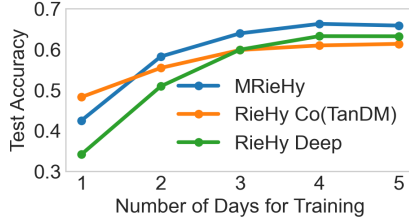
Figure 3: Neighbor vertices and incident hyperedges for a sample from Stieger2021 dataset Subject 1 in the covariance-based hypergraph component built with cosine similarity (left) and with TanDM (right). Numbers indicate the day each vertex is from.



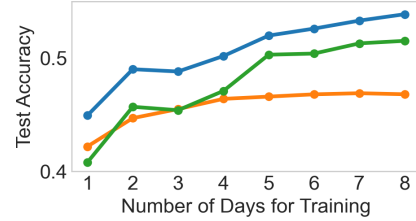
#### 4.6 Impact of the Number of Training Days

We study how training days affect test accuracy for MRieHy and its two components, as shown in Figure 4. On ECoG128, MRieHy’s accuracy rises markedly from one to three training days and then plateaus from four to five days; on Stieger2021, it improves steadily as more training days are added.

The two feature components exhibit distinct learning patterns. The deep feature-based component performs poorly with limited data but improves considerably when more data become available, whereas the covariance-based component, which relies on Riemannian tangent space distance to mean similarity, remains relatively stable across training days. This aligns with known characteristics of BaseNet’s data dependence and the stability of Riemannian MDM [31]. These results confirm that our tangent space similarity inherits Riemannian geometry’s stability, and that MRieHy’s multi-feature architecture effectively combines both feature types’ complementary strengths: covariance representations are especially valuable with limited training data, while deep features become advantageous with larger datasets.



(a) ECoG128 dataset



(b) Average of 7 subjects from Stieger2021 dataset

Figure 4: Impact of training day count on MRieHy and component hypergraph performance.

#### 4.7 Parameter Sensitivity

We further evaluate MRieHy’s sensitivity to three key hyperparameters: the number of neighbors  $k$  for hyperedge generation, the regularization weight  $\eta$  for multi-hypergraph fusion, and the online buffer size. Experiments were conducted on the BCI Competition IV 2a dataset (Figure 5 upper row) with default values  $k = 2$ ,  $\eta = 10000$ , buffer size = 32, and on the Stieger2021 dataset (Figure 5 bottom row) with defaults  $k = 5$ ,  $\eta = 500,000$ , buffer size = 32. In each analysis, one hyperparameter is varied while the others remain fixed.

As shown in Figure 5 left column, MRieHy’s accuracy stays relatively stable across different values of  $k$  on both datasets. For  $\eta$ , accuracy increases slightly as  $\eta$  grows on the BCI Competition dataset (Figure 5 upper middle) but remains stable on Stieger2021 (Figure 5 bottom middle). Finally, Figure 5 right column indicates that larger buffer sizes yield a small accuracy gain on the BCI Competition dataset, whereas on Stieger2021 accuracy increases marginally and then plateaus as the buffer grows. These results suggest that optimal hyperparameter combinations vary across datasets, yet overall performance remains robust to changes in these hyperparameters.

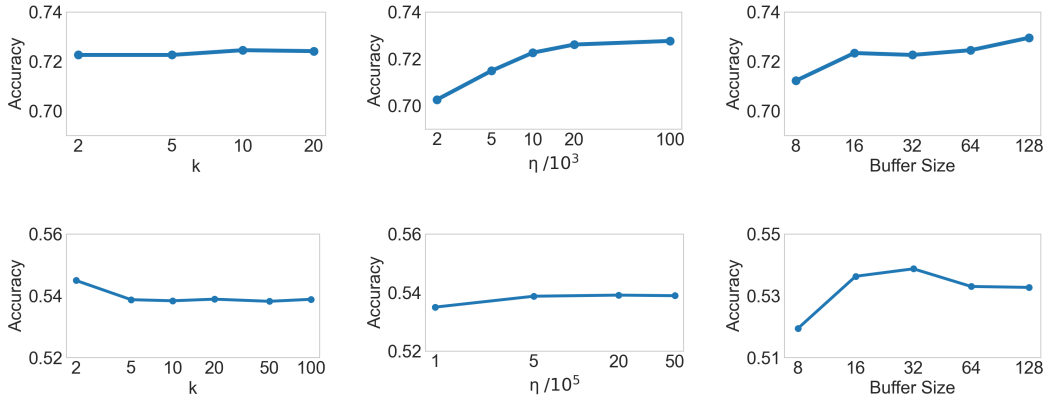


Figure 5: Effect of  $k$ ,  $\eta$ , and buffer size on MRieHy’s average test accuracy across 9 subjects on BCI Competition IV 2a (upper row) and 7 subjects on Stieger2021 (bottom row).

## 5 Conclusion

In conclusion, this paper presented the Multi-feature Riemannian Hypergraph (MRieHy), a novel framework for online test-time adaptation of MI-BCIs. By combining a Riemannian-distance-based hypergraph over covariance matrices with a cosine-similarity-based hypergraph over deep features, MRieHy effectively mitigates cross-day performance degradation without labeled test-day data. A limitation lies in the buffer sampling, which lacks stability analysis and outlier rejection mechanisms. Future work will explore more robust buffer management with anomaly detection for long-term plug-and-play BCI use.

## References

- [1] Alexandre Barachant, Stéphane Bonnet, Marco Congedo, and Christian Jutten. Multiclass brain–computer interface classification by riemannian geometry. *IEEE Transactions on Biomedical Engineering*, 59:920–928, 2012.
- [2] Laurent Bougrain, Sébastien Rimbart, Pedro Luiz Coelho Rodrigues, Geoffrey Canon, and Fabien Lotte. Guidelines to use transfer learning for motor imagery detection: an experimental study. *2021 10th International IEEE/EMBS Conference on Neural Engineering (NER)*, pages 5–8, 2021.
- [3] Shugeng Chen, Mingyi Chen, Xu Wang, Xiuyun Liu, Bing Liu, and Dong Ming. Brain–computer interfaces in 2023–2024. *Brain-X*, 2025.
- [4] Gernot Müller-Putz Alois Schlögl Clemens Brunner, Robert Leeb and Gert Pfurtscheller. Bci competition 2008–graz data set a, 2008.
- [5] Marco Congedo, Alexandre Barachant, and Rajendra Bhatia. Riemannian geometry for eeg-based brain-computer interfaces; a primer and a review. 2017.
- [6] Manuel Eder, Jiachen Xu, and Moritz Grosse-Wentrup. Benchmarking brain–computer interface algorithms: Riemannian approaches vs convolutional neural networks. *Journal of Neural Engineering*, 21, 2024.
- [7] Walaa H. Elashmawi, Abdelrahman Ayman, Mina Antoun, Habiba Mohamed, Shehab Eldeen Mohamed, Habiba Amr, Youssef Talaat, and Ahmed Ali. A comprehensive review on brain–computer interface (bci)-based machine and deep learning algorithms for stroke rehabilitation. *Applied Sciences*, 2024.
- [8] Reza Eyvazpour, Behraz Farrokhi, and Abbas Erfanian. A general model based on riemannian manifold for stable decoding movement trajectory from ecog signals. *iScience*, 2026.
- [9] He He and Dongrui Wu. Transfer learning for brain–computer interfaces: A euclidean space data alignment approach. *IEEE Transactions on Biomedical Engineering*, 67:399–410, 2018.
- [10] Demetres Kostas and Frank Rudzicz. Thinker invariance: enabling deep neural networks for bci across more people. *Journal of Neural Engineering*, 17, 2020.
- [11] Satyam Kumar, Hussein Alawieh, Frigyes Samuel Racz, Rawan Fakhreddine, and José del R. Millán. Transfer learning promotes acquisition of individual bci skills. *PNAS Nexus*, 3, 2024.
- [12] Dong-Hyun Lee. Pseudo-label : The simple and efficient semi-supervised learning method for deep neural networks. 2013.
- [13] Siyang Li, Ziwei Wang, Hanbin Luo, Lieyun Ding, and Dongrui Wu. T-time: Test-time information maximization ensemble for plug-and-play bcis. *IEEE Transactions on Biomedical Engineering*, 71:423–432, 2023.
- [14] Jian Liang, Ran He, and Tieniu Tan. A comprehensive survey on test-time adaptation under distribution shifts. *International Journal of Computer Vision*, 133:31 – 64, 2023.
- [15] Xiuyun Liu, Wen-Long Wang, Miao Liu, Ming-Yi Chen, Tânia Pereira, Desta Yakob Doda, Yu-Feng Ke, Shou-Yan Wang, Dong Wen, Xiao-Guang Tong, et al. Recent applications of eeg-based brain-computer-interface in the medical field. *Military Medical Research*, 12, 2025.
- [16] Yassine El Ouahidi, Giulia Lioi, Nicolas Farrugia, Bastien Padeloup, and Vincent Gripon. Unsupervised adaptive deep learning method for bci motor imagery decoding. *2024 32nd European Signal Processing Conference (EUSIPCO)*, pages 1626–1630, 2024.
- [17] Natasha M. J. Padfield, Jaime Zabalza, Huimin Zhao, Valentin Masero Vargas, and Jin Ma Ren. Eeg-based brain-computer interfaces using motor-imagery: Techniques and challenges. *Sensors (Basel, Switzerland)*, 19, 2019.

- [18] Tongjie Pan, Yalan Ye, Hecheng Cai, Shudong Huang, Yang Yang, and Guoqing Wang. Multimodal physiological signals fusion for online emotion recognition. *Proceedings of the 31st ACM International Conference on Multimedia*, 2023.
- [19] Tongjie Pan, Yalan Ye, Yangwuyong Zhang, Kunshu Xiao, and Hecheng Cai. Online multi-hypergraph fusion learning for cross-subject emotion recognition. *Information Fusion*, 108:102338, 2024.
- [20] Yiheng Peng, Jingjing Luo, Hongbo Wang, Shijie Guo, Yuzhu Guo, Dongsheng Xu, and Yang Li. Test-time adaptation for cross-subject motor imagery eeg classification using information-aggregation and source-guided weighting. *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–10, 2025.
- [21] Boris Reuderink, Jason D. R. Farquhar, Mannes Poel, and Anton Nijholt. A subject-independent brain-computer interface based on smoothed, second-order baselining. *2011 Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 4600–4604, 2011.
- [22] Steffen Schneider, Evgenia Rusak, Luisa Eck, Oliver Bringmann, Wieland Brendel, and Matthias Bethge. Improving robustness against common corruptions by covariate shift adaptation. *ArXiv*, abs/2006.16971, 2020.
- [23] P. Shenoy, Matthias Krauledat, Benjamin Blankertz, Rajesh P. N. Rao, and Klaus-Robert Müller. Towards adaptive classification for bci. *Journal of Neural Engineering*, 3:R13 – R23, 2006.
- [24] James R. Stieger, Stephen A. Engel, and Bin He. Continuous sensorimotor rhythm based brain computer interface learning in a large population. *Scientific Data*, 8, 2021.
- [25] Ryota Tomioka, Jeremy Hill, Benjamin Blankertz, and Kazuyuki Aihara. Adapting spatial filter methods for nonstationary bcis. 2006.
- [26] Carmen Vidaurre, Claudia Sannelli, Klaus-Robert Müller, and Benjamin Blankertz. Machine-learning-based coadaptive calibration for brain-computer interfaces. *Neural Computation*, 23:791–816, 2011.
- [27] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno A. Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- [28] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7191–7201, 2022.
- [29] Martin Wimpff, Bruno Aristimunha, Sylvain Chevallier, and Bin Yang. Fine-tuning strategies for continual online eeg motor imagery decoding: Insights from a large-scale longitudinal study. *2025 47th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pages 1–7, 2025.
- [30] Martin Wimpff, Mario Döbler, and Bin Yang. Calibration-free online test-time adaptation for electroencephalography motor imagery decoding. *2024 12th International Winter Conference on Brain-Computer Interface (BCI)*, pages 1–6, 2023.
- [31] Martin Wimpff, Jan Zerfowski, and Bin Yang. Tailoring deep learning for real-time brain-computer interfaces: From offline models to calibration-free online decoding. *ArXiv*, abs/2507.06779, 2025.
- [32] Guiying Xu, Zhenyu Wang, Honglin Hu, Xi Zhao, Ruxue Li, Ting Zhou, and Tianheng Xu. Riemannian locality preserving method for transfer learning with applications on brain-computer interface. *IEEE Journal of Biomedical and Health Informatics*, 28:4565–4576, 2024.
- [33] Yalan Ye, Tongjie Pan, Qianhe Meng, Jingjing Li, and Li Lu. Online eeg emotion recognition for unknown subjects via hypergraph-based transfer learning. In *International Joint Conference on Artificial Intelligence*, 2022.
- [34] Paolo Zanini, Marco Congedo, Christian Jutten, Salem Ben Said, and Yannick Berthoumieu. Transfer learning: A riemannian geometry framework with applications to brain-computer interfaces. *IEEE Transactions on Biomedical Engineering*, 65:1107–1116, 2018.
- [35] Zizhao Zhang, Haojie Lin, Xibin Zhao, R. Ji, and Yue Gao. Inductive multi-hypergraph learning and its application on view-based 3d object classification. *IEEE Transactions on Image Processing*, 27:5957–5968, 2018.
- [36] Sicheng Zhao, Guiguang Ding, J. Han, and Yue Gao. Personality-aware personalized emotion recognition from physiological signals. In *International Joint Conference on Artificial Intelligence*, 2018.
- [37] Dengyong Zhou, Jiayuan Huang, and Bernhard Scholkopf. Learning with hypergraphs: Clustering, classification, and embedding. In *Neural Information Processing Systems*, 2006.

## A Multi-feature Hypergraph Learning Details

Let  $\mathbf{Z} = [\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_n]$  be the concatenation of all sample features, where  $n$  is the total sample number for training.  $\mathbf{Z} \in \mathbb{R}^{C \times n}$  when using the covariance matrix as feature, and  $\mathbf{Z} \in \mathbb{R}^{d \times n}$  when using the deep feature. Let  $\mathbf{y}_i = \text{onehot}(y_i) \in \mathbb{R}^N$ , and let  $\mathbf{Y} = [\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n]^\top \in \mathbb{R}^{n \times N}$  denote the label matrix for all training samples, where  $y_i$  is the label of sample  $i$ . Following [35], the goal of hypergraph learning is to learn a regularized matrix  $\mathbf{M}$  to project the feature matrix  $\mathbf{Z}$  to the label space to discriminate different categories.

To learn the projection matrix  $\mathbf{M}$  for a single hypergraph, a cost function  $\Psi$  is introduced, aggregating the hypergraph Laplacian regularizer  $\Omega(\mathbf{M})$ , the empirical loss  $\mathcal{R}_{emp}(\mathbf{M})$ , and a regularization term on  $\mathbf{M}$  given by  $\Phi(\mathbf{M})$ :

$$\Psi = \{\Omega(\mathbf{M}) + \lambda \mathcal{R}_{emp}(\mathbf{M}) + \mu \Phi(\mathbf{M})\} \quad (13)$$

The hypergraph Laplacian regularizer for  $\mathbf{M}$  is under the assumption that vertices with strong connections should share similar labels. The hypergraph Laplacian regularizer is written as equation (14), and it takes a quadratic form in  $\mathbf{M}$ .

$$\Omega(\mathbf{M}) = \frac{1}{2} \sum_{k=1}^N \sum_{e \in \mathcal{E}} \sum_{u, v \in \mathcal{V}} \frac{\mathbf{W}(e) \mathbf{H}(u, e) \mathbf{H}(v, e)}{\delta(e)} \vartheta = \text{tr}(\mathbf{M}^\top \mathbf{Z} \Delta \mathbf{Z}^\top \mathbf{M}) \quad (14)$$

where  $\vartheta = \left( \frac{(\mathbf{Z}^\top \mathbf{M})(u, k)}{\sqrt{d(u)}} - \frac{(\mathbf{Z}^\top \mathbf{M})(v, k)}{\sqrt{d(v)}} \right)^2$ , and  $\Delta = \mathbf{I} - (\mathbf{D}^v)^{-\frac{1}{2}} \mathbf{H} \mathbf{W} (\mathbf{D}^e)^{-1} \mathbf{H}^\top (\mathbf{D}^v)^{-\frac{1}{2}}$ .

The empirical loss associated with  $\mathbf{M}$  is expressed as

$$\mathcal{R}_{emp}(\mathbf{M}) = \|\mathbf{Z}^\top \mathbf{M} - \mathbf{Y}\|^2 \quad (15)$$

To mitigate overfitting in  $\mathbf{M}$  while encouraging row-wise sparsity that highlights informative features, the  $\ell_{2,1}$  norm regularizer is employed, defined by

$$\Phi(\mathbf{M}) = \|\mathbf{M}\|_{2,1} \quad (16)$$

Putting these pieces together, the learning objective over the hypergraph is formulated as

$$\arg \min_{\mathbf{M}} \left\{ \text{tr}(\mathbf{M}^\top \mathbf{Z} \Delta \mathbf{Z}^\top \mathbf{M}) + \lambda \|\mathbf{Z}^\top \mathbf{M} - \mathbf{Y}\|^2 + \mu \|\mathbf{M}\|_{2,1} \right\} \quad (17)$$

Equation (17) can be solved by an iteration process. For the detailed derivation and solution steps, we refer the reader to [35].

In the context of multi-feature hypergraphs, the cost function  $\bar{\Psi}$  for learning the projection matrices  $\mathbf{M}_h$  from all hypergraphs comprises two components: the aggregated learning costs across individual hypergraphs and a regularization term applied to the combination weights  $\omega$ :

$$\bar{\Psi} = \sum_{h=1}^m \omega_h \{ \Omega(\mathbf{M}_h) + \lambda \mathcal{R}_{emp}(\mathbf{M}_h) + \mu \Phi(\mathbf{M}_h) \} + \eta \Gamma(\omega) \quad (18)$$

where  $m$  denotes the number of hypergraphs (equaling 2 in this research), and  $h$  indexes the specific hypergraph. In this formulation,  $\Gamma(\omega)$  is defined as the  $\ell_2$  norm of the modality weight vector:

$$\Gamma(\omega) = \|\omega\|^2 \quad (19)$$

This regularization term is designed to determine optimal modality-specific combination weights. The associated learning objective for the multi-hypergraph framework is formulated as:

$$\arg \min_{\mathbf{M}_h, \omega \geq 0} \left\{ \sum_{h=1}^m \omega_h \{ \Omega(\mathbf{M}_h) + \lambda \mathcal{R}_{emp}(\mathbf{M}_h) + \mu \Phi(\mathbf{M}_h) \} + \eta \Gamma(\omega) \right\}, \text{ s.t. } \sum_{h=1}^m \omega_h = 1 \quad (20)$$

Equation (20) decomposes into  $m + 1$  separable subproblems, each corresponding to a distinct projection matrix  $\mathbf{M}_h$  and the weight vector  $\omega$ . Consequently, the optimization proceeds by first solving for each  $\mathbf{M}_h$  independently with method similar to that for equation (13), followed by optimizing  $\omega$  to integrate the modalities.

With learned  $\mathbf{M}_h$ , the solution of the combination weight  $\omega_h$  is

$$\omega_h = \frac{1}{m} + \frac{\sum_{h=1}^m \Upsilon_h}{2m\eta} - \frac{\Upsilon_h}{2\eta} \quad (21)$$

where  $\Upsilon_h = \Omega(\mathbf{M}_h) + \lambda \mathcal{R}_{emp}(\mathbf{M}_h) + \mu \Phi(\mathbf{M}_h)$ . For detailed derivation, we also refer readers to [35].

## B Computational Cost

The experiments are conducted on NVIDIA GeForce RTX 3090 Ti GPU with 12th Gen Intel(R) Core(TM) i9-12900K CPU. The typical off-line training time of Multi-feature Riemannian Hypergraph is about 63ms per sample, and the on-line inference time is about 14ms per sample.

## C Ethics Statement

Experiments that contribute to this work were approved by IRB. All subjects consent to participate. All electrode locations are exclusively dictated by clinical considerations.

## D Percentage of Distinct Days per Hyperedge for More Subjects

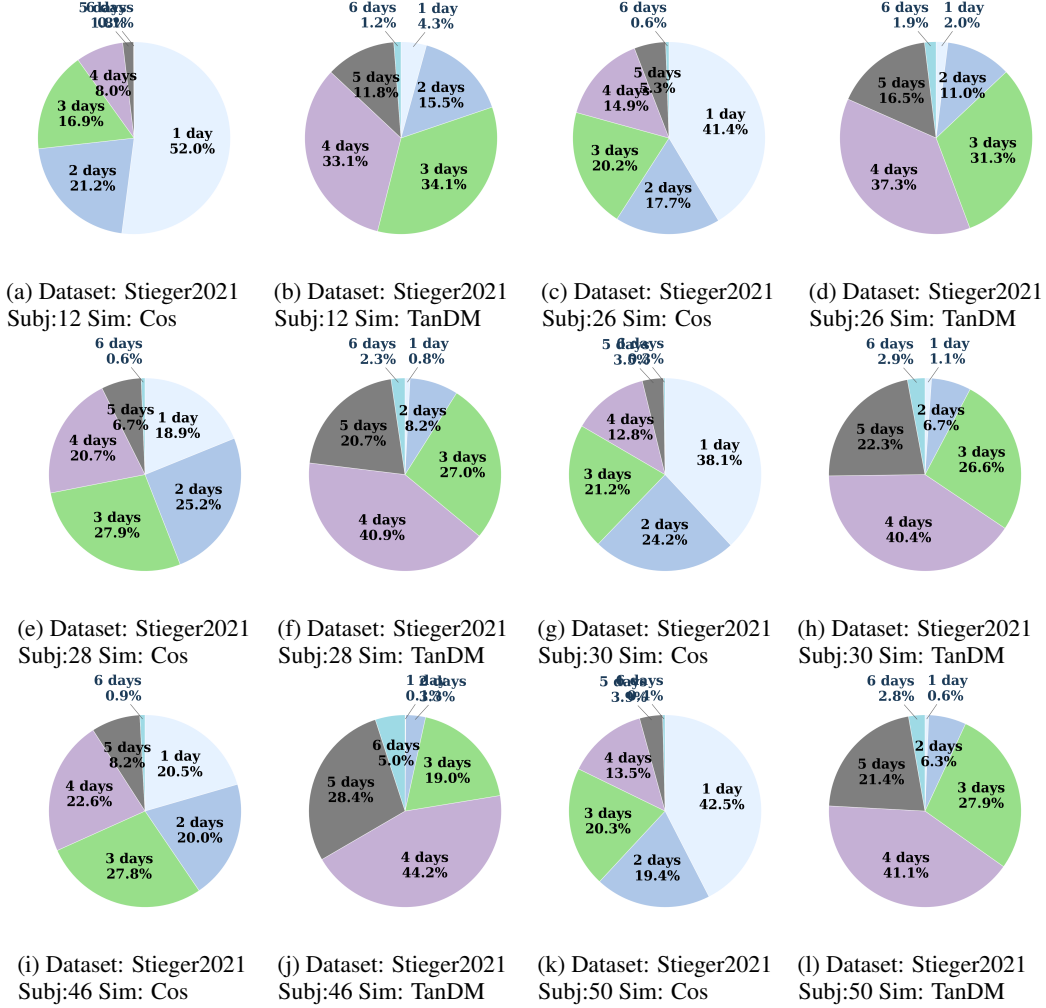


Figure D.1: Percentage of distinct days per hyperedge in the covariance-based hypergraph component